INDIVIDUAL THESIS

ADAPTIVE SCHEDULING AND GENERATIVE PROJECT PLANNING

Intelligent Project Management System for Undergraduate Students

Research group R26-IT-095

Lekamge R.L.M.D.

(IT22117014)

Dissertation submitted in partial fulfilment of the requirements for the

BSc (Hons) in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology (SLIIT)

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

A draft AI-assistance disclosure is provided in Appendix B. The candidate must verify it against the module policy before signing this declaration.

Student name: Lekamge R.L.M.D

Student ID: IT22117014

Candidate signature: 

Date: 10/11/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Prof. Samantha Rajapaksha	_____	_____
Co-supervisor: Ms. Kaushika Kahatapitiya	_____	_____

ABSTRACT

Undergraduate project teams must turn proposals into task plans, allocate work, inspect evidence and arrange academic evaluation events. This dissertation examines the Adaptive Scheduling and Generative Project Planning component of the Intelligent Project Management System (IPMS). Its parser, effort estimator, dependency-aware Gantt generator, evidence records and academic timetable form an evidence-aware prototype. The live workload transition uses Longest Processing Time (LPT) assignment and persists a separate balanced schedule version. An isolated-PostgreSQL regression run on 11 October 2026 passed 11 focused checks, including hall-capacity enforcement, schedule-history member restoration and a synthetic local PDF approval and signed-webhook path. In 100 simulated historical-profile trials, LPT averaged 3.72% planned-workload CV, while a historical classifier-guided hybrid averaged 4.40%. A legacy mixed-label JIRA row split reported 81.27% accuracy, but a chronological Sourcetree-only proxy test achieved 31.94%, below its 77.31% majority baseline; neither result measures student suitability. Rule extraction recovered 73/75 listed tasks in synthetic proposals, not general extraction accuracy. The effort regressor's log MAE was 0.8554 and its measured original-hour MAPE about 152.6%. The contribution is a tested local planning workflow with explicit evidence boundaries, not demonstrated improvements in real-cohort outcomes.

Keywords: proposal-driven scheduling; workload balancing; progress evidence; academic timetabling; XGBoost.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, **Prof. Samantha Rajapaksha**, and co-supervisor, **Ms. Kaushika Kahatapitiya**, for their valuable guidance, constructive feedback, and continuous support throughout this research project. Their advice was important in refining the research problem, methodology, evaluation process, and development of the Intelligent Team Formation and Topic Feasibility Analysis component.

I also extend my appreciation to the academic staff of the Sri Lanka Institute of Information Technology for providing the knowledge, resources, and research environment required to complete this work.

I am grateful to the students who participated in the data collection process and provided the competency and project-preference information used in the real-response case study. Their participation contributed directly to the practical evaluation of the proposed framework.

I would also like to thank my research group members for their cooperation in developing and integrating the complete **Intelligent Project Management System for Undergraduate Students**. Their collaboration was important in ensuring that the individual components could operate together as an integrated solution.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
ABSTRACT ii
ACKNOWLEDGEMENTS iii
TABLE OF CONTENTS iii
LIST OF FIGURES vi
LIST OF TABLES vii
LIST OF ABBREVIATIONS viii
1 INTRODUCTION 1
1.1 Background 1
1.2 Problem statement and research gap 1
1.3 Aim and objectives 2
1.4 Scope and individual contribution 2
1.5 Research questions 3
1.6 Rationale for an individual-component report 3
1.7 Stakeholders and decision boundaries 4
1.8 Design requirements and success criteria 4
1.9 Structure of the dissertation 5
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Intelligent project planning 6
2.2 Proposal extraction and task evidence 6
2.3 Effort, dependencies and workload allocation 6
2.4 Evidence and academic timetabling 6
2.5 Comparison of the building blocks 7
2.6 Resulting research gap 8
2.7 Why proposal extraction is not ordinary summarisation 8
2.8 Effort prediction and the baseline problem 9
2.9 Activity evidence versus completed work 9
2.10 Preferences and hard constraints in academic timetabling 10
2.11 Synthesis 10
3 RESEARCH METHODOLOGY 11
3.1 System architecture and data flow 11
3.2 Input validation and proposal-grounded extraction 12
3.3 Total effort and task-level effort 12
3.4 Dependency-aware Gantt scheduling 13
3.5 Work classification and ownership 13
3.6 Experimental proxy model, LPT and fairness 13
3.7 Progress, documents and signed GitHub evidence 14
3.8 Academic event scheduler 15
3.9 Input and output contract 15
3.10 Why the scheduler uses two assignment notions 16
3.11 Scheduling mathematics and a worked illustration 17
3.12 Balancing mathematics and interpretation 18
3.13 Document and repository evidence state machine 18
3.14 Academic-event decision procedure 19
3.15 Scenario exploration versus operational decisions 20
3.16 Ethical and privacy design considerations 20
3.17 Extraction precedence, thresholds and provenance 21
3.18 Feature construction and domain shift 21
3.19 Fairness objective and indivisible tasks 22
3.20 Evidence-state rules and permitted transitions 23
3.21 Event-search policy and unresolved constraints 23
4 PRODUCT AND SOLUTION DESIGN 25
4.1 Principal files and interfaces 25
4.2 Two-stage schedule state 25
4.3 Persistence and access control 25
4.4 Example operational sequence 25
4.5 API boundaries and component responsibilities 26

4.6 Persistence model and traceability 27
4.7 User-facing workflow and accessibility considerations 28
4.8 Failure handling and recoverability 28
4.9 Scope of the implemented optional models 29
5 IMPLEMENTATION, INTEGRATION AND TESTING 30
5.1 A proposal-to-task execution trace 30
5.2 Document-review transaction 30
5.3 Webhook trust and task matching 31
5.4 Academic draft lifecycle 31
5.5 Deployment configuration and boundaries 32
5.6 Authoritative auto-assignment route 32
5.7 Data sources and experimental boundaries 33
5.8 Workload-model validation 33
5.9 Controlled allocation comparison 34
5.10 Functional and integration test cases 34
5.11 Proposal-extraction test protocol 35
5.12 Model-evaluation protocol 36
5.13 Allocation-baseline reproducibility 36
5.14 Evidence and academic constraint tests 37
5.15 What the reported tests do not establish 38
5.16 Recommended acceptance criteria for a later field pilot 38
5.17 Reproducibility of the predictive experiments 38
5.18 Independent outputs for the allocation comparison 39
5.19 Traceability tests for the document pathway 40
5.20 Webhook and event-scheduler negative cases 40
5.21 Visual and document review protocol 41
5.22 Evidence status and reporting rule 41
6 RESULTS AND VALIDATION 42
6.1 Predictive and extraction results 42
6.2 Allocation baselines 43
6.3 Operational findings 44
7 DISCUSSION 45
7.1 Interpreting the workload classifier 45
7.2 Why the hybrid requires constraints 45
7.3 Extraction and evidence examples 46
7.4 Academic scheduling and integration interpretation 46
7.5 Threats to validity 47
7.6 Error analysis of the classifier 48
7.7 Interpreting the fairness numbers 48
7.8 Interpreting the extraction result 49
7.9 Component-level answer to the research questions 49
7.10 Summary of individual contribution 51
8 COMMERCIALISATION AND SUSTAINABILITY 52
8.1 Value proposition and target users 52
8.2 Adoption and evaluation path 52
8.3 Data rights, governance and sustainability 52
9 CONCLUSION AND FUTURE DIRECTIONS 53
9.1 Summary of achieved objectives 53
9.2 Limitations and next research steps 53
REFERENCES 55
APPENDIX A: EVIDENCE AND CODE MAP 57
APPENDIX B: ACADEMIC INTEGRITY AND AI-USE DISCLOSURE 58

LIST OF FIGURES

Figure 3.1. Implemented planning workflow; LPT is the live workload assignment policy.11
Figure 3.2. Document evidence is submitted and reviewed separately from signed repository activity. 23
Figure 6.1. Historical-profile allocation baseline: mean workload CV over 100 paired trials. 43
Figure 7.1. Legacy mixed-label row-split confusion matrix; rows are actual labels and columns are predictions. 48

LIST OF TABLES

Table 1.1. Functional requirements and verification points. 2
Table 2.1. How the integrated component relates to the methods reviewed. 7
Table 3.1. Workload-model features. 14
Table 3.2. Academic panel and hall constraints. 15
Table 3.3. Planning and academic-scheduling input contract. 16
Table 3.4. Evidence semantics and limits. 19
Table 4.1. Main API families and their boundaries. 26
Table 4.2. Selected persisted entities and their audit roles. 27
Table 5.1. Datasets and intended evaluation roles. 33
Table 5.2. Integration and test-case matrix; expected behaviour, not a claim that every row was independently rerun for this document. 34
Table 6.1. Observed numerical outputs and the exact bounds of their interpretation. 42
Table 6.2. Historical-profile and cold-start allocation baselines over 100 paired trials per scenario. 43
Table 7.1. Additional historical-profile allocation measures (100 trials). 45
Table A.1. Evidence and code map 57

LIST OF ABBREVIATIONS

Term	Meaning
API	Application Programming Interface
CV	Coefficient of Variation
DAG	Directed Acyclic Graph
FP	Function Points
GitHub HMAC	Hash-based Message Authentication Code for webhook signatures
LPT	Longest Processing Time first
NSGA-II	Non-dominated Sorting Genetic Algorithm II
PDF	Portable Document Format
PP1 / PP2	Progress Presentation 1 / 2
TF–IDF	Term Frequency–Inverse Document Frequency
WBS	Work Breakdown Structure
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Background

A student research proposal explains objectives, methods and expected outputs, but it is rarely a ready-to-execute schedule. Supervisors still need to identify deliverables, estimate effort, place work in dependency order and decide who will perform each task. As work proceeds, documents and code activity must be inspected rather than accepted as self-reported percentages. The same project later requires Proposal, PP1, PP2 and Viva sessions involving academic staff, rooms and time windows. These coupled decisions motivate a planning component that connects the first proposal upload to the final evaluation snapshot.

The component studied here is one part of IPMS, research group R26-IT-095. Its design goes beyond a visual Gantt: a task can retain a proposal source, planned duration, dependency position, assigned owner and later progress evidence. The live assignment route uses LPT, while classifier-guided ownership is only an offline comparison. Planning and judgement remain separate: computed outputs do not replace academic review.

1.2 Problem statement and research gap

A manually prepared plan can be inconsistent when a long proposal contains shared activities, individual component work and parallel tasks undertaken by every member. A naïve model-only allocator can repeatedly select the same high-scoring member, producing an unworkable distribution. A static schedule can also conceal what changed after the original plan, while a commit count can be mistaken for completed work. Existing work on AI-enabled project management identifies opportunities for forecasting and decision support [1], but it does not by itself resolve these practical integration and audit problems for an undergraduate academic workflow.

The research problem is therefore: how can a proposal-grounded planning system create feasible dates, defensible task ownership, verifiable progress records, and conflict-aware academic events without treating uncertain predictions or repository activity as final academic decisions?

1.3 Aim and objectives

The aim is to develop and evaluate a scheduling component that turns project proposals into auditable plans and continues tracking those plans through academic assessment.

1. Extract actionable WBS tasks and group identifiers from uploaded proposal PDFs while retaining source evidence.
2. Estimate effort, validate dependencies, and calculate working-day task dates for a Gantt view.
3. Separate shared, parallel and individual work; use LPT for live ownership and evaluate classifier-guided alternatives only offline.
4. Store versioned schedules and link document submissions and signed GitHub events to task progress.
5. Generate Proposal, PP1, PP2 and Viva timetables with eligible, available panels and halls.
6. Evaluate model behaviour, controlled allocation fairness, extraction recovery and integration tests, reporting limitations honestly.

1.4 Scope and individual contribution

This report covers the scheduling component's Python/FastAPI backend, separate test submission portal, PostgreSQL schema, Gantt interface, offline workload-policy evaluator and academic-event functions. Other IPMS components such as quality assessment, risk management and team formation provide context but are not claimed as this component's independent implementation. The contribution is a traceable planning-and-review workflow, its LPT-based live workload transition and bounded offline measurements. It does not claim that historical JIRA ranking improves student assignments.

Table 1.1 presents functional requirements and verification points.

Table 1.1. Functional requirements and verification points.

Requirement	Verification point
Proposal-derived WBS	Tasks retain section/evidence and reject unsuitable text
Feasible project dates	DAG validation, working-day calculation, no exposed temporary owner

Requirement	Verification point
Balanced ownership	Live LPT route, version persistence and rollback in focused database tests; offline LPT/hybrid comparison
Progress evidence	Task-key mapping, signed webhooks, document review states
Academic events	Role, availability, overlap and hall-capacity checks under a stated seat policy
Auditability	Versioned schedules, evidence records and Viva snapshots

1.5 Research questions

The component was examined through three bounded questions. RQ1 asks how a JIRA-derived outcome proxy and four allocation policies behave on historical issue fixtures; it does not test whether the classifier identifies the best student for a task. RQ2 asks whether an evidence-preserving extractor can recover explicitly listed deliverables from controlled proposal PDFs. RQ3 asks which planning, review and academic-timetabling states are represented in the prototype and which constraints remain unverified. These questions do not assert that the system produces optimal or fully deployed schedules.

RQ1 is addressed descriptively by legacy and chronological proxy metrics and simulated allocation baselines, all subject to domain and label-validity limits. RQ2 is addressed by three known-WBS proposal fixtures. RQ3 combines source contracts with six focused database-backed regression tests for the live assignment and academic-capacity paths. These tests are not a full-system or real-cohort evaluation. The study does not measure long-term student productivity or compare outcomes against an institutional control cohort.

1.6 Rationale for an individual-component report

The two sample dissertations use a recognisable academic sequence: context and gap, methodology, implementation, test-case design, results, discussion, conclusion and references. This report follows that sequence while concentrating on one component of a larger group system. It explains where this component receives project data, how it generates a task plan and how it returns schedules and review states to the wider IPMS dashboard. It does not reproduce unrelated group components as if they were developed or experimentally validated within this individual study.

That distinction matters in an integrated prototype. A scheduling screen may display team-formation information or a quality score that originates elsewhere. This dissertation treats those as upstream context or downstream integration unless a scheduling route itself calculates the value. The core scientific claim is therefore an evidence-aware scheduling workflow and a measured allocation policy, not ownership of every model visible in the complete portal.

1.7 Stakeholders and decision boundaries

The component has three principal user groups. A project supervisor needs a readable plan, an explanation of each proposed assignment and a way to accept or challenge submitted evidence. Academic staff need a cross-group view of upcoming evaluations, panel roles and room availability. Students need only the limited submission path for their own document tasks; they do not need access to the planning controls, other groups' files or timetable publication. These different responsibilities explain the decision to keep the submission portal separate from the staff-facing application. They also define what a successful output means: it must be useful to the right actor at the right point in the project lifecycle, not merely calculable by the server.

The generated WBS and dates are proposals for review. A supervisor may decide that a task is too broad, that an effort estimate is unrealistic or that a specialist should own work despite the balance preference. A GitHub commit is even weaker evidence: it records repository activity, not that the deliverable is correct. By contrast, a review decision is an explicit human judgement linked to a particular submission. This report uses the terms estimate, suggestion, recorded activity and approval deliberately. Treating them as interchangeable would make the dashboard misleading and would weaken both academic accountability and any later evaluation of the algorithms.

1.8 Design requirements and success criteria

The first requirement is provenance. Every task shown after parsing should retain enough source information for a reviewer to locate the supporting proposal passage. The second is schedule validity: predecessor references must be meaningful, cycles must be rejected, and generated dates must use a declared working-day convention. The third is fairness transparency. Shared work must remain visible as shared, per-

member parallel work must not be collapsed into a single owner, and individual assignments must be accompanied by member totals and a comparable baseline. The fourth is evidence integrity: files, review decisions and webhook events must have distinct identities and auditable timestamps. The fifth is timetabling feasibility: a published event must not knowingly reuse unavailable staff or a booked hall interval.

The current prototype enforces a stated hall-seat policy in draft generation, rescheduling and publication. That is an implementation check, not proof that every institution's attendance policy is represented. Likewise, extraction recovery on three deliberately structured PDFs shows the pipeline works on those fixtures, not that it will parse scanned or unfamiliar proposals. The evaluation chapter distinguishes implemented behaviour, tested behaviour and future validation needs.

1.9 Structure of the dissertation

Chapter 2 analyses the relevant approaches and explains why no single model solves the full project lifecycle. Chapter 3 describes the data, equations and staged methodology. Chapter 4 maps those decisions onto concrete APIs, storage and user journeys. Chapter 5 states the experiments and integration checks, including their sampling boundaries. Chapter 6 presents the measured outputs and examines where the prototype is strong or uncertain. Chapter 7 concludes with a deployment-oriented research agenda. This sequence mirrors the supplied individual-report examples while avoiding repetition of work performed by the other IPMS components.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Intelligent project planning

The literature on AI-enabled project management reports a growing role for predictive models and decision support [1]. A prediction, however, must be assessed with its dataset and error measure. Software-effort estimation research has repeatedly shown that model rankings are sensitive to the evaluated data and to how errors are reported [2]. For this component, the effort model provides a planning estimate, not a guaranteed number of work hours. The Gantt risk indicator likewise expresses a capacity ratio, not a calibrated failure probability.

2.2 Proposal extraction and task evidence

Research on requirements classification and scientific information extraction offers methods for finding useful statements in lengthy text [3], [4]. T5 provides a general text-to-text architecture [5], and a T5-small option is present locally. The operational path in this component instead uses proposal headings, WBS-like patterns and filtering rules because these preserve a direct source excerpt for each accepted task. A small zero-shot T5 check did not establish sufficient reliability to make it the default extractor.

2.3 Effort, dependencies and workload allocation

The effort regressor and offline issue-status classifier use boosted decision trees implemented with XGBoost [6]. A project is not schedulable from effort alone: tasks may depend on earlier work, so the component validates a directed acyclic graph and uses a stable topological order [7]. LPT assigns larger tasks to the least-loaded member and is both the live default and the fairness baseline [8]. The classifier resembles historical recommendation work [9], but its proxy score has not been validated as student suitability. The guarded classifier hybrid is measured only by a separate offline evaluator.

2.4 Evidence and academic timetabling

GitHub activity is an imperfect proxy for completed engineering work [10]. A commit linked to a WBS task is useful evidence of activity, but a supervisor must still inspect the deliverable. For academic events, topic similarity ranks potential panel members

through TF–IDF cosine similarity, with an optional MiniLM path [11], [12]. Room, role and overlap constraints are checked separately; this follows the broader distinction between preference scoring and feasible timetabling [13]. NSGA-II and Monte Carlo capabilities are available for scenario analysis [14], [15], but they are not presented here as proven optimisers of student outcomes.

2.5 Comparison of the building blocks

Table 2.1 presents how the integrated component relates to the methods reviewed.

Table 2.1. How the integrated component relates to the methods reviewed.

Existing approach	Strength	Remaining need in this study
Predictive effort models	Fast estimate from historical project features	Link estimate to proposal tasks and expose uncertainty
Requirements/WBS extraction	Reduces manual reading	Retain source evidence and reject academic noise
LPT scheduling	Simple and reproducible load distribution	Combine fairness with any useful suitability signal
GitHub mining	Timestamped development activity	Separate activity evidence from approval
Academic timetabling	Formal feasibility constraints	Connect events to the same project identity and progress state

The first design tension is between predictive preference and capacity. An unconstrained proxy-score policy can concentrate work, while a load-only policy does not consider expertise. The offline hybrid evaluator permits a load guard to override a high score, but it does not show better expertise matching. LPT is the deliberate live default: the controlled results show it is a strong fairness baseline and a defensible cold-start policy until student-labelled evidence supports a learned alternative.

The second tension is between automation and inspectability. A generated task name alone does not tell a supervisor whether it was actually mentioned in the proposal. A risk badge alone does not reveal whether it is a probability or a simple ratio. Similarly, a green GitHub event might be only a successfully delivered webhook rather than evidence that a project milestone passed review. The component addresses these ambiguities by exposing source sections, calculation rules, review states and stored event records.

2.6 Resulting research gap

The reviewed methods provide useful pieces—text extraction, effort prediction, constrained scheduling, fair assignment and event auditing—but none of them independently gives the entire undergraduate-project lifecycle considered here. The gap is operational and methodological: turning a proposal into a dated, owned and reviewable plan while preserving the boundary between calculated indicators and academic judgement. The present artefact tackles this gap through explicit stages and a single project identifier. It is evaluated as a prototype, not claimed as a universally optimal scheduling algorithm.

2.7 Why proposal extraction is not ordinary summarisation

A prose summary of a proposal and an executable work breakdown structure serve different purposes. A summary can preserve the overall meaning while omitting small but schedulable activities; a WBS needs a separate record for each action that consumes time or yields an inspectable output. For example, 'design and evaluate a prototype' is a useful summary, but it does not indicate whether requirements, interface implementation, testing and evaluation depend on one another. The extractor therefore prioritises explicit WBS rows, then consults Gantt entries, specific objectives and action-oriented methodology prose when coverage is weak. This ordered strategy trades some linguistic flexibility for source traceability. It is related to requirements extraction [3], [4], but its operational target is a project task record with a schedule identity, not a generic text category.

The source-method confidence attached to a task is a rule-based priority marker. A WBS row receives a higher value than a task inferred from methodology prose because the former is usually a more direct planning statement. Those values are not probabilities learned from annotated proposals and have not been reliability-calibrated. A dissertation that displays 0.95 beside a WBS task must therefore explain that it means 'high-priority extraction source' rather than a measured 95% chance that the task is correct. Optional T5 generation [5] could produce more fluent candidates, but fluent language does not solve provenance. Its five-example zero-shot check was too small and weak to replace the deterministic path.

2.8 Effort prediction and the baseline problem

Software-cost estimation reviews [2] warn against interpreting a point estimate without its error distribution or comparator. In this system, the regressor predicts project-level log effort from sparse size information, then rules distribute that total over extracted tasks. This is a practical mechanism for creating an initial schedule, but it compounds two uncertainties: the project-level hours may be wrong, and the allocation weights may not reflect the actual complexity of individual tasks. The measured log-scale errors are therefore central results. The evaluation script's exp(log MAE)-1 calculation is not a measured percentage-error metric. The proper use of the output is a revisable first estimate with visible assumptions.

The same baseline principle applies to workload allocation. LPT is a strong, explainable fairness comparator [8] and the default live allocator. A JIRA-trained outcome proxy cannot be called a student-suitability model without student-assignment labels. In offline simulations, a guarded hybrid approached but did not beat LPT on workload fairness; no later quality or completion outcome was measured. The historical classifier is therefore reported as a limited research experiment, not the deployed assignment model.

2.9 Activity evidence versus completed work

Repository mining is attractive because GitHub events have timestamps, actors and immutable identifiers. However, a commit can be trivial, unrelated to a task or authored by someone other than the member who ultimately explains the work. The cautions in repository-mining literature [10] are directly relevant: available digital traces are not a complete behavioural record. This system adds two layers of interpretation. First, the webhook must be authenticated and associated with a registered repository and task key. Second, even a valid match contributes only an activity signal. An authorised human must still inspect the code or submitted document before concluding that a task has met its academic requirement.

Document submissions create a stronger review path because the submitted artifact can be opened and the decision recorded against its version. They are not intrinsically trustworthy either: the file may be incomplete, mismatched or superseded. This is why the state transition separates receipt from approval and preserves negative decisions

with reviewer comments. The combination of source-linked plan, event-linked activity and decision-linked completion is the central evidence-aware aspect of the component. It gives the dashboard a way to show progress without pretending every machine-generated signal has the same evidential weight.

2.10 Preferences and hard constraints in academic timetabling

Academic panel selection combines two kinds of information. Topic similarity is a preference: it ranks eligible staff but cannot make an unavailable person available. Role composition, interval availability, booking conflicts and hall-seat capacity are hard constraints: a high similarity score cannot compensate for their violation. This separation follows general timetabling practice [13]. The generated batch remains a draft until staff inspect and publish it. The seat requirement derives from a configured team-and-audience policy, which must still be checked against actual institutional attendance rules.

2.11 Synthesis

The reviewed approaches motivate a staged architecture. Explicit task evidence comes before effort and dates; feasibility rules limit predictive preferences; versioned state preserves what was proposed; and human review determines academic acceptance. None of these steps removes uncertainty. Their value lies in making uncertainty visible and allowing a supervisor to correct a suggestion without losing the original evidence trail. The quantitative evaluation should therefore be read as evidence about particular stages under declared data conditions, not as a single accuracy number for the entire IPMS platform.

3 RESEARCH METHODOLOGY

3.1 System architecture and data flow

The user supplies a proposal PDF, group ID, team size, expected months, start date and function points. PDF extraction and validation create a candidate WBS; an effort regressor and task-type weights assign provisional hours. Dependencies and internal capacity lanes produce dates, while visible ownership remains Unassigned. An authenticated second stage classifies shared, parallel and individual work, applies LPT to individual tasks and saves a distinct balanced schedule version. Evidence and academic-event modules are separate from the historical classifier experiment.

Figure 3.1 shows implemented planning workflow; lpt is the live workload assignment policy.

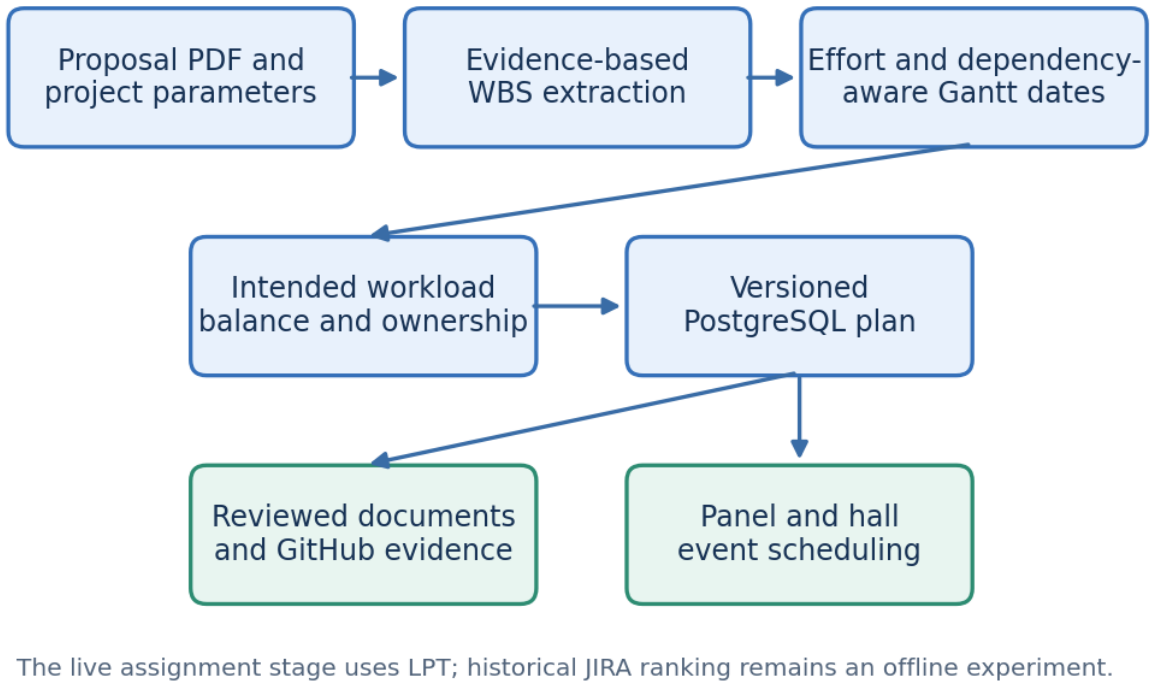


Figure 3.1. Implemented planning workflow; LPT is the live workload assignment policy.

FastAPI exposes the workflows as REST endpoints. PostgreSQL database ipms_scheduling stores schedule versions, assignments, progress, submissions and academic events. The staff/supervisor application uses role-checked actions, while

students use a separate submission portal for eligible document tasks. This separation limits student access to administrative functions.

3.2 Input validation and proposal-grounded extraction

The proposal upload route reads PDF text and rejects a file that lacks sufficient meaningful project content. The group ID is normalised to a stable format before any schedule is stored. The extractor searches headings, bullet-like entries and implementation-oriented phrases, then removes duplicates, front matter, reference items and fragments that do not describe executable work. Each retained task carries a source section, extraction method, source text, confidence and validation reason. These fields make it possible to challenge an unsuitable generated task instead of accepting an opaque model output.

The operational extractor is deliberately conservative. A T5-small model can optionally rewrite text as candidate tasks, but it is disabled by default. The controlled evaluation uses three synthetic proposals with 25 explicit WBS entries each. This tests recovery under known conditions; it does not establish general accuracy on arbitrary research PDFs.

3.3 Total effort and task-level effort

The project-level XGBoost regressor receives function points, log-transformed function points and a source indicator. It predicts log effort; inverse transformation converts the result back to hours. The result is clipped to a capacity range derived from team size and expected months. Each task is classified into a broad lifecycle type such as requirements, model development, integration or documentation, and a complexity level. Its effective weight is the product of type and complexity weights. Task i receives approximately $E_i = \max(4, \hat{E} \times w_i / \Sigma w_j)$, where $\hat{E}$ is clipped project effort and w_i is the task's effective weight. A rounding correction is applied to the final task so the allocated hours remain close to the predicted total.

The model was evaluated on 580 China and Desharnais project records, with 116 rows held out. Its log-scale MAE is 0.8554 and log RMSE is 1.0614. The former script's 135.2% value equals exp(log MAE)-1; it is not a measured MAPE and should not be labelled relative prediction error. An independent original-hour calculation on the

same hold-out gives about 152.6% MAPE and 61.8% median absolute percentage error. At inference, the source feature is fixed to 0, so the training source distinction is not observed for a new student project. These limitations make the estimate suitable only as a revisable planning starting point.

3.4 Dependency-aware Gantt scheduling

The scheduler builds a DAG over task indices, rejects invalid predecessor identifiers and cycles, and returns a stable topological order. For each task, the earliest start is the later of project start, the first working day after all predecessors finish, and the free date of an internal capacity lane. It skips weekends and calculates an end date from productive hours per day. The saved response is returned in stable task-ID order, even though dates were calculated in dependency order. This process is dependency-aware but does not calculate critical-path slack or CPM metrics.

A task's displayed risk uses required working days divided by available working days before the project deadline. Ratios below 0.70 are LOW, 0.70–0.89 are MEDIUM and at least 0.90 are HIGH. This is a capacity heuristic rather than a learned or calibrated probability of failure. The separate saved delay experiment is not the source of this Gantt label.

3.5 Work classification and ownership

The workload stage begins after Gantt date generation. It classifies shared tasks as All Members, creates per-member representations for parallel activities, and reserves a single owner for individual tasks using LPT. A common report therefore remains visibly shared rather than appearing as one student's sole responsibility. Focused database tests check that the live transition stores this ownership in a new schedule version; the classification itself still requires review on varied real proposals.

3.6 Experimental proxy model, LPT and fairness

The legacy XGBoost classifier uses seven features: task effort proxy, task category, task complexity, assignee outcome aggregate, current open-count aggregate, average effort and a collaborative flag. Class 1 means Fixed for Sourcetree but not-delayed for Apache; these are incompatible proxies rather than student outcomes. The artifact named workload_rf.joblib contains an XGBClassifier despite stale Random Forest

metadata. A separate evaluator compares random, LPT, score-only and hybrid policies. The live API uses LPT by default; neither the legacy artifact nor the newer Sourcetree-only chronological proxy is validated for student suitability.

Table 3.1 presents workload-model features.

Table 3.1. Workload-model features.

Feature	Operational interpretation
Task effort estimate	Expected task hours
Task category	Bug/backend-like versus other work
Task complexity	Low / medium / high ordinal value
Member completion rate	Sourcetree Fixed / Apache not-delayed aggregate; training rows include self-outcome
Current workload	Existing open-task/load indicator
Member average effort	Historical mean task effort or default
Collaborative flag	Whether work is shared; constant in current training set

In the offline hybrid evaluator, selecting the highest proxy score without restriction concentrates work. A pre-assignment guard compares the selected member's existing load with 1.40 times the least load and may redirect a task. It does not simulate adding the task first and thus is not a hard per-task guarantee. A subsequent local search evaluates whole-task moves using overload excess, max–min spread and variance in that order. CV = 100 × standard deviation(load) / mean(load) reports planned-effort dispersion; lower CV does not establish better expertise matching or work quality. These experimental hybrid operations must not be attributed to the live LPT route.

3.7 Progress, documents and signed GitHub evidence

Every scheduled task has a stable key such as R26-IT-095-TASK-024. The separate test-only submission portal lists eligible document tasks for a group and student ID. A tester can submit PDF or DOCX evidence against an assigned task. The integration checks the task, file type, size, assignment and duplicate hash, and stores a versioned submission. The review state is PENDING_REVIEW until an authorised supervisor/staff reviewer opens the file and chooses APPROVED, REJECTED or CHANGES_REQUESTED, with comments required for negative decisions. Approval can complete a document task; a mere upload cannot. This portal is disabled in production mode pending institutional student authentication.

A group may register a GitHub repository. Push, pull-request and review events are accepted only after HMAC signature verification with the configured webhook secret. A task key gives a direct task match; ambiguous events remain flagged rather than silently completing work. A commit may start partial progress, but code activity alone does not prove task completion. Progress summaries weight each task's verified percentage by planned effort; the Viva snapshot freezes the state for later academic inspection.

3.8 Academic event scheduler

Academic staff upload a supervisor/panel workbook containing role, expertise and available date/time data, plus a hall workbook with location, capacity and availability. Topic–expertise similarity ranks candidates, while a panel must contain two Professors/Doctors, one Lecturer and one Instructor. Auto-scheduling checks distinct membership, staff and hall interval conflicts, opening windows, project event-week offsets and required seating capacity. The seat policy uses team size, four panel members and configurable additional audience seats. If no valid combination is found, the event is unscheduled with a reason. Drafts can be inspected, published, rolled back, rescheduled and exported; publication and rescheduling recheck hall capacity.

Table 3.2 presents academic panel and hall constraints.

Table 3.2. Academic panel and hall constraints.

Constraint	Required behaviour
Panel roles	Two Professor/Doctor, one Lecturer, one Instructor
Panel uniqueness	No person appears twice in one panel
Availability	Every member and hall available for full event interval
No overlap	No conflicting staff/hall booking
Room suitability	Opening hours, interval conflicts and required seat capacity checked
Event sequence	Proposal, PP1, PP2 and Viva follow configured weeks

3.9 Input and output contract

A planning workflow is only reproducible when its input contract is explicit. The request binds a project/group identifier to the proposal text, start date, number of students, expected duration and function-point estimate. The identifier is important because schedule history, individual progress, academic events and external evidence

all use it to refer to the same project. Team-member IDs are extracted from the proposal where possible; they are not inferred from an unrelated project. Invalid identifiers, an empty useful WBS or invalid dependency references are rejected rather than silently replaced with a generic demonstration project.

Table 3.3 presents planning and academic-scheduling input contract.

Table 3.3. Planning and academic-scheduling input contract.

Input	Validation / transformation	Output that depends on it
Proposal PDF	Readable text and proposal-content checks	Tasks, task source evidence, research topic
Group ID	Pattern extraction and normalisation	Version/history and evidence join key
Start date	Parsed calendar date; moved to next working day if needed	Gantt task dates
Team size / member IDs	Positive size and distinct IDs	Capacity lanes, workload owners
Expected months	Positive duration; may be increased by detected proposal Gantt span	Project horizon and capacity cap
Function points	Numeric size estimate	Total effort-regressor feature
Staff/hall sheets	Required columns, dates, times, roles and capacities	Feasible academic-event draft

The response is more than a bar position on the Gantt. It includes task ID and stable key, task name, proposal source, extraction method, estimated hours, derived duration, start/end dates, dependency IDs, task category, complexity, visible owner, assignment stage, document requirement, capacity-risk label and progress state. Keeping this record complete allows a later review screen to explain where a percentage or warning came from. The scheduler stores the version as JSON while related evidence is kept in dedicated tables, so the original task plan and later review activity can be examined separately.

3.10 Why the scheduler uses two assignment notions

The date-generation algorithm must respect limited capacity even before the system knows final owners. It therefore gives each task a temporary internal lane when selecting its earliest free date. A temporary lane is a computational resource, not a commitment by a named student. Earlier implementations exposed this lane as the task owner, which made a newly generated Gantt appear fully allocated before the workload model had run. The current response explicitly sets assignment_stage to

UNASSIGNED and assigned_to to Unassigned until the balancing endpoint completes its own decision. This is not merely a visual correction: the history snapshot must represent the same state that a supervisor saw at generation time.

The second assignment uses persisted tasks, member IDs, existing work and task classification. Shared work contributes to every member's load; parallel work creates separate entries; individual tasks receive one owner selected by LPT. The route saves a balanced version transactionally and rejects stale or repeated operations. Focused database-backed tests exercise the saved ownership and rollback path. Historical proxy scores are not used for the default live selection.

3.11 Scheduling mathematics and a worked illustration

Let e_i be planned effort in hours and h be the productive-hour allowance per working day. The implementation computes $d_i = \max(1, \text{ceil}(e_i / h))$; in the current generation route h is 4.0. If task B depends on task A, B cannot begin before the first working day after A ends. Its candidate start is max(project start, predecessor-ready date, lane-ready date), moved forward if it falls on a weekend. This is a precedence-and-capacity schedule, not a critical-path calculation: no latest-start dates or float values are calculated.

For illustration only, suppose an extracted task requires 20 hours, h = 4 hours/day and its predecessor completes on a Thursday. The duration becomes five working days. If its lane is free on Friday, the task can start Friday and finish the following Thursday, assuming no public-holiday calendar is configured. If the lane is not free until Tuesday, the start moves to Tuesday. This example explains the date rule; it is not an observed result from the R26-IT-095 database. It also shows why project schedules can differ from naïvely dividing all hours by the full team size.

The schedule-risk badge is a separate calculation. Let a_i be working days available between task start and the project deadline. The displayed utilisation is $u_i = d_i / \max(1, a_i)$. The LOW/MEDIUM/HIGH thresholds are 0.70 and 0.90. For example, a five-day task with ten working days available has u = 0.50 and is LOW; the same task with five days available has u = 1.00 and is HIGH. The 100% shown in the latter case means

full utilisation of available time, not a statistically estimated 100% probability of failure.

3.12 Balancing mathematics and interpretation

In the offline evaluator, one proxy feature vector is formed for each task-member pair. The score-only policy chooses the highest XGBoost class score; the hybrid policy may override it when the selected member's pre-assignment load exceeds 1.40 times the least load. The candidate task's hours are not included in that initial comparison. A later local search accepts only improving whole-task moves. Its sampled trials had no overload violations, but sufficiently coarse tasks can make the threshold infeasible. This hybrid policy is not the live LPT assignment rule.

$CV = 100\sigma(L)/\mu(L)$ reports the relative spread of member loads. Consider the illustrative four-member planned loads 100, 105, 95 and 100 hours. Their mean is 100, population standard deviation is about 3.54, and CV is about 3.54%. The max/min ratio is about 1.11, below the 1.40 overload threshold. CV and the ratio capture related but different questions. CV measures overall dispersion, while the guard focuses on an extreme disparity. The report uses both rather than treating a single fairness score as sufficient evidence of good ownership.

A model can appear confident in a cold start because it receives default member-history values, but equal defaults provide no real member-specific evidence. The live LPT rule and the offline evaluator's load guard remain useful fairness mechanisms in that situation. Cold-start trials show that planned-hour balance does not depend entirely on stored profiles. They do not establish that the selected student is most qualified for a specialised task; that judgement requires project-specific evidence and supervisor review.

3.13 Document and repository evidence state machine

A document task moves from planned/not started to submitted, then to pending review. Approval can set its verified progress to completion; rejection or requested changes retain a reviewable record and require comments. The implementation stores the original filename, storage path, SHA-256 digest, MIME type, file size, submission version, reviewer, decision and timestamps. The default maximum document size is

20 MB, configurable through an environment variable. File signatures are checked in addition to extensions, reducing the chance that a disguised upload is accepted. A duplicate digest for the same project and task is rejected. These controls do not replace malware scanning or a production retention policy.

For GitHub, the trust boundary begins at the webhook endpoint. The raw request body is authenticated using the configured secret and the X-Hub-Signature-256 HMAC header before it is interpreted. Deliveries and commit SHAs have uniqueness controls to prevent accidental double counting. A repository must be registered to a project, and task references follow a stable key format such as [R26-IT-095-TASK-024]. Actor mapping links a GitHub login to a student identity when configured. An event that lacks a valid match remains visible as rejected or flagged evidence instead of being silently assigned to the nearest task title. A linked commit can raise partial progress but does not mark the task complete.

Table 3.4 presents evidence semantics and limits.

Table 3.4. Evidence semantics and limits.

Evidence	What it establishes	What it does not establish
Document uploaded	A versioned file was received for a task	That its contents meet academic requirements
Document approved	An authorised reviewer accepted that submission	That unrelated tasks are complete
Signed push event	The event came through the configured GitHub secret	That code is correct or finished
Task-keyed commit	Repository activity can be linked to a task	That the named author did all the work
Viva snapshot	The recorded progress state at capture time	That every percentage is externally audited

3.14 Academic-event decision procedure

Event scheduling begins with a configured project start date and week offsets for Proposal, PP1, PP2 and Viva. The service loads the relevant group/topic, active staff and active halls. Staff topic scores are computed, but selection is constrained by the required role combination. A candidate is feasible only if the panel and a sufficiently large hall are free for the full interval. Required seats include team members, four panellists and configured audience seats. Previously booked intervals are considered

for the next group. If no feasible combination exists, the event is not forced into a conflicting or undersized room.

A generated academic timetable is a draft. This matters because workbook quality and policy choices vary by institution. Staff can inspect scheduled and unscheduled counts, compare versions, publish a chosen batch, roll back or reschedule a particular event. Each such action is separate from the initial algorithmic suggestion. The component also exposes calendar and Excel views for practical administration. The test workbooks are intentionally synthetic; a real staff list and hall policy would need validation by the faculty before operational use.

3.15 Scenario exploration versus operational decisions

The code contains a Monte Carlo route that samples 1,000 possible schedule paths for a hypothetical intervention and an NSGA-II route that searches objective trade-offs involving duration, burnout/workload and milestone visibility. These endpoints are useful for demonstrating what-if analysis and multi-objective alternatives. They are not part of the measured historical-profile allocation baseline and have not been rigorously benchmarked against an accepted optimum. The dissertation therefore documents their implementation role while reserving its quantitative conclusions for the evaluated extractor, workload classifier, effort regressor and paired allocation experiment.

3.16 Ethical and privacy design considerations

An individual proxy score can easily be misunderstood as a judgement of a student's ability. The legacy artifact was trained on mixed issue-tracker outcomes, with only five Sourcetree assignee identifiers and five Apache project pseudo-members. A chronological Sourcetree-only rerun also performed below its majority baseline. Neither is validated on students and neither should become an academic grade or a basis for excluding a student from a learning task. The live LPT allocation remains reviewable by supervisors; any override should be recorded with its workload effect.

Submitted reports, names, student IDs and GitHub logins are personal data. A production version needs a defined purpose, minimum necessary collection, consent where applicable, restricted access by group, a retention period and a deletion

procedure. The webhook secret and submission integration key should be environment secrets rather than values committed to a repository. The prototype's test files are synthetic; real staff or student documents should not be copied into a public demonstration site. These considerations are part of the method's feasibility, not optional cosmetic work.

3.17 Extraction precedence, thresholds and provenance

The extractor does not run all methods indiscriminately. It first looks for an explicit work breakdown section and parses numbered rows one line at a time. This line boundary prevents a regular expression from joining one row to the next when PDF text extraction has flattened a table. If fewer than eight accepted tasks remain, the algorithm consults a Gantt section and then specific objectives. Only if fewer than four valid tasks remain does it search action verbs in methodology and solution prose. The optional language-model fallback is attempted at the same weak-coverage threshold and is disabled in the default configuration. These thresholds are heuristics that favour an explicit WBS over speculative task creation; they are not trained parameters.

Task filtering removes candidate titles that fail structural checks, duplicated names and fragments associated with references, abbreviations or front matter. A retained task records an extraction method and, when available, the supporting sentence or row. The source scores are fixed metadata: WBS rows, Gantt entries, objectives, methodology actions and fallback candidates have different configured priorities. They should be interpreted as provenance strength, not calibrated confidence. A useful future experiment would blind annotators to these scores and measure whether each source tier actually predicts acceptance by an academic reviewer. Until then the interface should show the source itself alongside any score.

3.18 Feature construction and domain shift

The workload classifier's 1,411 rows are issue-tracker records, not students. The Sourcetree subset has five named assignee IDs; the Apache subset is represented by five project-level pseudo-members. The binary target is not uniform: Sourcetree class 1 means Resolution == Fixed, whereas Apache class 1 means delayed == 0. Training-only aggregation avoids directly calculating a test feature from its own target, but each

training row contributes its own outcome to its assignee's completion-rate and open-count features. Random splitting can also place future observations of the same assignee in the profile used for an earlier held-out issue. Therefore the 81.27% result describes this particular row split, not leakage-free prospective performance or student suitability.

The legacy XGBoost artifact can score issue-derived feature vectors offline, but its class probability is not a calibrated student-performance confidence. It reflects a mixed JIRA target, not skill, quality or expected completion date. A newer Sourcetree-only chronological experiment removes mixed labels and constructs as-of histories, yet reaches only 31.94% accuracy on 216 later issues, below the 77.31% majority baseline. Consequently the live auto-assignment route defaults to deterministic LPT; the historical hybrid is retained only for offline comparison.

3.19 Fairness objective and indivisible tasks

The offline hybrid evaluator uses planned effort L_m for each member. Its pre-assignment guard compares the model-selected member's existing load with 1.40 times the least existing load; it does not simulate the current task first. A local improvement phase tests whole-task moves and prefers lower overload excess, then lower maximum-minus-minimum spread, then lower variance. These are properties of the historical experimental policy. The live route instead uses LPT with final whole-task overload validation and must not be described as executing the classifier-based hybrid.

The algorithm moves whole tasks because a software component or report section may need one accountable owner. This makes the optimisation discrete. For example, if all members already carry similar loads but the only remaining task is much larger than the others, no one-step assignment can keep every max/min ratio below 1.40. The measured zero-violation result applies to the sampled evaluation trials, not every conceivable proposal. A production workflow should flag an infeasible target and offer a staff decision, possibly splitting the task into valid subtasks after reviewing its meaning, rather than silently reporting an impossible fairness guarantee.

3.20 Evidence-state rules and permitted transitions

Figure 3.2 shows document evidence is submitted and reviewed separately from signed repository activity.

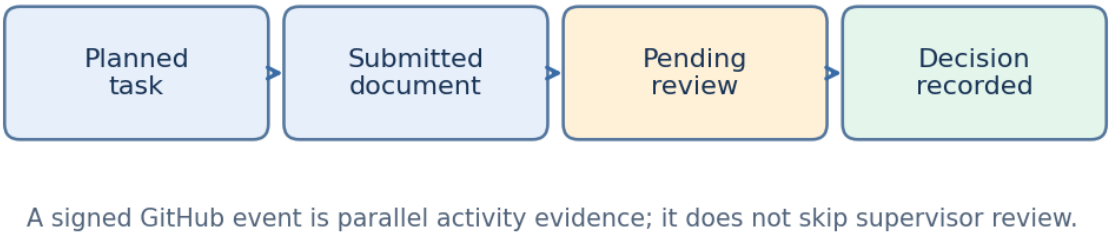


Figure 3.2. Document evidence is submitted and reviewed separately from signed repository activity.

A task's planning state, activity evidence and review state are different dimensions. A planned task can have no evidence. A submitted document acquires a pending-review state, with its hash and version preserved. An approval may raise verified progress to completion for a document task. A rejection or changes-requested decision retains the submission and comment so a replacement can be evaluated later. Signed repository activity is stored alongside this sequence, but it does not jump directly from 'commit received' to 'approved'. This separation also means a Viva snapshot can faithfully show a partially progressed task without asserting that it passed review.

3.21 Event-search policy and unresolved constraints

The academic route uses week offsets from the project start date for the four event types. It searches candidate days after a target date, advances in half-hour slot increments, and accepts a configured duration between 30 and 240 minutes. Candidate panels must have the required role pattern; topic similarity is adjusted by a panel-load penalty so the same academics are not always preferred. The route verifies staff and hall availability over the complete interval and checks conflicts with existing bookings. The current candidate-day search is bounded rather than an unlimited global optimiser, so an event can be reported unscheduled even if a more distant date might work.

Hall capacity is now loaded with candidate rooms and checked against required seats before an event is accepted. The configured policy combines team size, the four panel members and additional audience seats; publication and manual rescheduling also reject undersized rooms. A database-backed regression test covers this rule. Capacity alone cannot establish full suitability: institutional holidays, room equipment and actual attendance still need local policy and independent review.

4 PRODUCT AND SOLUTION DESIGN

4.1 Principal files and interfaces

The main orchestration routes and the Gantt/workload calculations are in component_scheduling/main.py. Model preparation is in train_workload_model.py; allocation baseline evaluation is in evaluate_workload_balancing.py. Progress, submissions, webhook evidence and Viva snapshots are handled by progress_tracking.py. The academic panel-role and interval helpers are in academic_scheduler_service.py.PostgreSQL persistence is provided through postgres_db.py. Browser-facing interfaces are in frontend/, while the student submission application is separate under student_submission_portal/. These are local implementation locations, not separate claims of model authorship.

4.2 Two-stage schedule state

The generate endpoint constructs a dated Gantt whose displayed owner is Unassigned; internal lanes are only a date-planning device. The authoritative auto-assignment route then classifies shared, parallel and individual work, applies LPT to individual tasks and persists a separate balanced version. The historical classifier-guided hybrid remains a separate offline experiment. Focused database-backed tests check that the live route responds, saves ownership and version history, and rolls back on failure.

4.3 Persistence and access control

A generated schedule is saved as a new PostgreSQL version rather than replacing the previous baseline. History can reopen the version and its ownership state. Progress events, document submissions, review decisions and academic batches are stored separately, allowing a reviewer to see what was planned, what was submitted and what was approved. Student-facing submission actions use a separate integration key; supervisor and staff operations are role-protected, and reviewer-to-group mappings are checked when configured. The system should still be deployed only after environment secrets, repository permissions and document retention policies are reviewed.

4.4 Example operational sequence

1. Staff/supervisor uploads a valid proposal and checks the extracted group ID and WBS evidence.

2. The backend estimates total effort, assigns task weights, validates dependencies and creates a versioned dated schedule.
3. A tested balancing action determines shared, parallel and individual ownership and persists per-member load in a new version.
4. Students submit task-linked documents through the separate portal; staff inspect and review those files.
5. Signed GitHub events are linked or flagged, then progress summaries and Viva snapshots are refreshed.
6. Academic staff upload panel/hall workbooks, generate event drafts, resolve any unscheduled cases and publish the timetable.

4.5 API boundaries and component responsibilities

The proposal, schedule and workload routes live in the main backend. The progress router adds document-submission, review, evidence and Viva-specific operations. Academic routes include supervisor and hall upload, schedule creation, batch management and exports. This route separation mirrors a domain separation: a schedule calculation is not a document judgement, and a document judgement is not an academic event booking. The frontend calls the corresponding backend route and displays the returned state; it should not manufacture a different progress percentage from local UI assumptions.

Table 4.1 presents main api families and their boundaries.

Table 4.1. Main API families and their boundaries.

Route family	Principal responsibility	Important control
/api/schedule/*	Proposal upload, Gantt generation and history	Proposal validation; versioned save
/api/progress/auto-assign	Live LPT-based authoritative ownership	Role check, version guard, transaction and rollback
/api/progress/submissions/*	Document read and review	Role/group check; decision audit
/api/integrations/*	Standalone student portal connection	Integration key; assigned-task filter
/api/webhooks/github	Signed repository activity	HMAC, repo and delivery validation
/api/academic/*	Panel, hall and event-batch workflow	Role, availability and overlap constraints

An HTTP success response should also be interpreted narrowly. A successful proposal upload confirms that a file was parsed and a task set was returned; it does not mean the extracted plan was accepted by the supervisor. A successful GitHub webhook delivery confirms authentication and receipt; it does not mean a task passed evaluation. This distinction is reflected in separate statuses and event records. It is especially important in a live demonstration, where the green check in GitHub's Recent Deliveries refers to transport success, not academic approval.

4.6 Persistence model and traceability

The active database adapter is PostgreSQL, configured by SCHEDULING_DATABASE_URL and normally targeting ipms_scheduling. The adapter translates legacy placeholder syntax used by older routes into PostgreSQL-compatible parameter binding. It does not write application data to the earlier SQLite file described in an outdated README. Schema creation includes schedule_versions for Gantt snapshots, task_assignments and task_assignment_history for ownership, task_progress and progress_events for current and historical state, and task_submissions for uploaded evidence. Additional tables store registered repositories, mapped GitHub identities, deduplicated deliveries, commits and pull requests.

The academic side stores active supervisors and halls, their availability windows, event panel members, versioned schedule batches and audit entries. Separating these records allows each subsystem to maintain its own invariants. For example, a schedule version can remain available even after a document is rejected; a reviewer decision can be traced without altering the original proposal text; a published academic batch can be distinguished from a later draft. The database transaction around a version or review action prevents partially updated state from appearing as a completed operation.

Table 4.2 presents selected persisted entities and their audit roles.

Table 4.2. Selected persisted entities and their audit roles.

Entity	Key fields	Why it is retained
schedule_versions	project_id, version, schedule_json	Original and later Gantt states
task_assignments	project_id, task_id, assigned_to, effort	Current authoritative ownership

Entity	Key fields	Why it is retained
task_progress / progress_events	task_key, status, percentage, actor, time	Current progress and change history
task_submissions	task_key, version, hash, review_status	Document provenance and decision
github_commits	repository, SHA, task_key, identity	Deduplicated code-activity evidence
academic_schedule_batches	version, status, configuration	Draft/published timetable states
academic_event_audit	event, action, actor, before/after	Rescheduling and publication trace

4.7 User-facing workflow and accessibility considerations

The planning interface presents proposal input alongside the generated Gantt, and its workload panel shows member totals, CV, overload verdict and explanations. The task-detail dialog exposes effort, duration, risk and evidence for inspection. The live LPT endpoint supplies the balanced ownership; no historical classifier score is presented as a validated student recommendation. The separate test submission page is narrower: testers provide group and student IDs, load eligible document tasks and upload files; staff-facing review and publication controls remain separate.

This UI separation is also a usability requirement. A large group list needs search by group ID, status filtering and compact summaries before opening individual task details; otherwise a single-group design becomes impractical with many teams. Colour-coded event types and risk badges are supplemented by text labels because colour alone is ambiguous and inaccessible to some viewers. The report does not claim formal accessibility certification or usability-study results; these are design choices that can be tested in a later evaluation.

4.8 Failure handling and recoverability

Several failure paths are implemented. The live assignment route uses LPT as its standard policy and rolls back an incomplete database transaction. The academic service has a TF–IDF fallback if optional embeddings are unavailable; the dependency scheduler rejects cycles; the event search can leave an unmatchable event unscheduled. Document and webhook handlers include file-cleanup and signature checks. Focused database-backed tests cover selected assignment and capacity failures, but the remaining paths still need broader service-level verification.

Recoverability also depends on versioning. Repeating schedule generation creates a new version, allowing a supervisor to compare it with the earlier baseline. Workbook uploads may mark previous staff or hall rows inactive, so a test workbook should only be used in a test database or followed by re-uploading the authoritative institutional list. A quick Cloudflare tunnel used during development is not a production availability or security mechanism. Deployment would require a stable HTTPS endpoint, secured secrets, database backups and explicit operational ownership.

4.9 Scope of the implemented optional models

The codebase contains names inherited from earlier prototypes. The workload artifact filename includes rf and its metadata says Random Forest, but the saved estimator is XGBClassifier. Older README descriptions of SQLite and semantic GitHub matching also do not describe the current audited path. The training script, serialized estimator, route order, PostgreSQL adapter and signed task-key webhook router are examined separately rather than assuming a later function definition is reachable. The auxiliary logistic-risk experiment is not the source of displayed Gantt risk; T5-small, MiniLM, Monte Carlo and NSGA-II remain optional or exploratory here.

5 IMPLEMENTATION, INTEGRATION AND TESTING

5.1 A proposal-to-task execution trace

Consider the local R26-IT-095 proposal used as an integration fixture. When its PDF is uploaded, the backend first extracts text and checks that the document contains sufficient proposal-like content. It identifies the group key and attempts to recover actionable work from explicit sections. The returned task record contains a stable identifier, title, source section and method, so the staff view can compare an item with the proposal before accepting it. Effort estimation and dependency scheduling then create a versioned Gantt. At this point the rendered owner is Unassigned even if an internal lane was used to place the bar on the timeline. This is the state the initial history version must reproduce.

The second stage is an authenticated auto-assignment request that reads the active schedule version and member IDs, separates shared, parallel and individual work, then persists ownership and progress in a new version. The live route uses LPT rather than the historical issue-status classifier. Database-backed tests check route uniqueness, owner persistence, version creation and rollback on failure; the experiment does not establish improved real-student outcomes.

5.2 Document-review transaction

The separate test submission page requests the project key and student ID, then asks the backend for eligible document tasks. A file upload binds one task key to a byte stream. The server enforces a size limit, checks PDF or DOCX signatures as well as the extension, computes a SHA-256 digest and rejects an unchanged duplicate for the same task. It records a version with pending review. This portal supports demonstration without exposing staff planning controls; it is disabled in production mode because self-entered identifiers are not institutional student authentication.

A reviewer opens the stored version and records one of the supported decisions. The decision changes the review state and creates an audit entry rather than replacing the underlying file. Rejection and changes-requested responses require comments because the student needs a specific reason to revise the artifact. An approved document can complete the corresponding document task, while an upload by itself cannot. This

distinction prevents the progress chart from rewarding a submission that has not been checked. A failed database operation removes a newly stored file where the code can safely do so, reducing the chance of a dangling file with no review record.

5.3 Webhook trust and task matching

GitHub integration has two setup steps that are easy to confuse. Registering a repository URL associates a project with a source repository. Creating a webhook in GitHub points that repository at a reachable HTTPS endpoint and supplies the same secret configured in the backend. For each delivery, the backend verifies an HMAC over the raw body before trusting any parsed actor, commit message or task key. A green Recent Deliveries icon on GitHub shows only that the endpoint accepted the HTTP delivery. The application must still verify the registered repository and map a stable task key such as [R26-IT-095-TASK-024] to a saved task.

The task-key convention is intentionally explicit. A weak textual match between 'API work' in a message and 'API Integration' in the Gantt could attach evidence to the wrong team member or project. A stable key narrows that ambiguity and permits a reviewer to inspect the exact commit. The event's SHA and delivery identifier support deduplication. If the event is valid but has no task key, its activity can remain visible as unmatched rather than being forced into a task. If the signature is invalid, it is not trusted evidence at all. The UI should display these states in words, not only as a coloured badge or rising percentage.

5.4 Academic draft lifecycle

For academic scheduling, workbook upload and event generation are separate operations. The workbook parser normalises staff roles and availability and stores hall names and time windows. A generation request loads the latest group/topic data, applies the configured weeks and event duration, ranks eligible staff by topic similarity with load adjustment, and searches for a non-overlapping panel and hall interval. The result is a draft batch with scheduled and unscheduled events. A staff member can inspect an event, change it where the interface allows, and publish the chosen version. That human publication step is materially different from generating candidate times.

The current route checks hall capacity against the selected seat policy during draft generation. Rescheduling and publication recheck the rule, so a time-compatible but undersized hall is rejected. A database-backed regression test covers an undersized room. Staff must still confirm actual attendance, accessibility and equipment before publication; the numerical seat policy is not a substitute for institutional room approval.

5.5 Deployment configuration and boundaries

The backend depends on a reachable PostgreSQL instance and correctly configured database URL. The student portal depends on the scheduling backend being available, which is why a connection-refused response when loading tasks is a service-startup problem rather than a missing frontend style. GitHub push evidence additionally depends on a configured webhook secret and a public endpoint reachable from GitHub. During local testing a temporary tunnel can supply the latter, but its URL and uptime are not stable. The system must not display local 127.0.0.1 as if GitHub can call it from the Internet.

A production deployment would require a stable domain and TLS termination, secret management, backups, access logs, a documented file-retention policy and an institutional identity provider. These are not measured achievements of the prototype. They are prerequisites for real student documents and academic schedules, especially because the database joins group IDs, member IDs, submitted files and repository identities. The dissertation keeps this boundary clear so that successful local integration is not mistaken for security certification or production readiness.

5.6 Authoritative auto-assignment route

The application now registers one POST /api/progress/auto-assign route. Staff or supervisors submit a valid group and member set; the route validates persisted tasks, applies shared/parallel/individual classification, allocates individual work with LPT and saves a distinct balanced schedule version in one transaction. A project-scoped database lock and stale-version check prevent overlapping or repeated balancing from silently replacing the active plan. Focused database-backed tests cover the route, saved ownership and rollback. They do not constitute a real-cohort effectiveness study.

5.7 Data sources and experimental boundaries

Table 5.1 presents datasets and intended evaluation roles.

Table 5.1. Datasets and intended evaluation roles.

Data source	Size	Purpose and boundary
Sourcetree Jira CSV (GFG_FINAL.csv)	49,000 raw; 1,078 used	Assigned, dated issue-status proxy; five assignees
Local Apache-project summary	333 prepared rows	Issue-derived proxy; origin not row-verifiable
China + Desharnais local merge	499 + 81 projects	Effort regression; China pairs checked vs. Zenodo [18]
Synthetic proposal PDFs	3 / 75 tasks	Known-WBS extraction fixtures
Synthetic panel and hall sheets	30 staff / 8 halls	Parser and constraint fixtures
Progress fixture	5 documents / 13 commits	Evidence integration checks; not a user study

The workload experiment reads two local files. Despite its legacy filename, GFG_FINAL.csv contains 49,000 Sourcetree-for-Windows (SRCTREEWIN) Jira rows; filtering for an assignee and parseable creation date leaves 1,078 issues from five assignee IDs. A matching issue key is visible in Atlassian's public tracker [17], but the exact bulk-export query, acquisition date and reuse terms are not recorded. The second file, apache_jira_delay.csv, contains 333 prepared rows bearing five Apache project labels. It lacks issue keys, source URLs and an extraction manifest, so its row-level origin and transformation cannot be reconstructed; project codes serve as pseudo-members, not people. The effort file combines 499 China records and 81 Desharnais records. The China adjusted-function-point/effort pairs match all 499 records, in order, in Zenodo's China v1 release [18]. The Desharnais pairs match the local 81-row Desharnais file and a publicly available PROMISE-labelled copy, but the exact acquisition route is undocumented [16]. These are software-project proxies, not student outcomes. SciREX, world-cities, panel sheets and prepared commit/document fixtures are not inputs to these evaluated models. Redistribution rights still require a separate provider check.

5.8 Workload-model validation

The 1,411 records were divided with a stratified 80/20 row split using seed 42. The estimator uses 300 trees, learning rate 0.05, depth 5 and row/column subsampling of

0.8. The label is mixed: Sourcetree class 1 is Fixed resolution, Apache class 1 is not delayed. Member aggregates are computed from training rows, but each training row contributes its own target to its completion-rate and open-count features. The same assignee or pseudo-member can appear in both split portions, and future outcomes can enter a supposedly historical profile. Five-fold CV reuses aggregates prepared before its folds. Consequently, the reported accuracy and CV describe this split only; neither is a leakage-free prospective measure of student-assignment performance.

5.9 Controlled allocation comparison

The allocator baseline used 100 paired trials for each scenario. Every trial sampled 20 distinct tasks from 283 held-out issue rows and allocated them to four members. Historical-profile trials drew four of ten stored profiles; cold-start trials gave all members the same default profile. Random, LPT, model-only and hybrid policies received the same task sets. The measures were workload CV, maximum–minimum hour spread and the proportion of trials violating the 1.40 load ratio. This is controlled resampling of issue data, not observation of 100 real student teams.

5.10 Functional and integration test cases

Table 5.2 presents integration and test-case matrix; expected behaviour, not a claim that every row was independently rerun for this document.

Table 5.2. Integration and test-case matrix; expected behaviour, not a claim that every row was independently rerun for this document.

Case	Input/action	Expected outcome
T01	Upload valid proposal PDF	Group ID and traceable WBS tasks returned
T02	Upload unsuitable PDF	Validation error; no misleading schedule saved
T03	Provide cyclic dependency	422/error and no invalid Gantt dates
T04	Generate schedule	Versioned dates; visible owners Unassigned
T05	POST authenticated auto-assign	One LPT handler saves owners, loads and a new version (focused DB test)
T06	Submit document twice unchanged	Second identical file rejected by hash
T07	Approve/reject submission	Decision and comments audited; progress handled by rule
T08	Send bad-signature webhook	Rejected; no evidence accepted
T09	Send signed task-key push	Linked activity evidence; no automatic completion
T10	No feasible staff/hall slot	Event unscheduled with reason, not double-booked
T11	Open older schedule after balancing	Stored original and balanced versions remain distinct (focused DB test)
T12	Freeze Viva snapshot	Stable reviewed progress state saved

An older audit mentions 60/60 backend tests, but its raw run log was not retained and is not counted as current evidence. A focused run on 11 October 2026 passed 11 tests against the isolated PostgreSQL database ipms_scheduling_codex_test_20261010. It covered the unique authenticated LPT route, versioning and rollback, production demo-auth refusal, hall-seat rejection, schedule-history member IDs, mixed-format deadline scans and one synthetic local PDF submission → supervisor approval → signed task-key webhook journey. The fixture report separately verifies 30 panel records, eight hall records, 13 prepared task-keyed commits and five example documents. These checks do not establish real student authentication, a new GitHub.com webhook delivery, cross-component payload integration or improved academic outcomes.

5.11 Proposal-extraction test protocol

The extraction test starts with three synthetically authored PDFs, each containing a numbered list of 25 known WBS tasks and a distinct group identifier. For each PDF, the evaluator first checks that the parser recovers the intended group ID. It then compares the accepted task names and their source evidence with the explicit list, counts how many entries were returned and inspects any omissions. This is a recall-style fixture check, not a precision estimate over arbitrary prose: a separate annotation exercise would be needed to label all non-task sentences and measure false-positive extraction comprehensively.

A useful negative test is a PDF that looks plausible at the file level but is not a software-project proposal—for example, a résumé or an unrelated administrative letter. The validation route checks meaningful proposal content before creating a plan. A second negative test uses a proposal with a group ID but no actionable WBS; it should not generate generic tasks merely to populate a chart. When a long proposal contains page numbers, references, repeated headings and broad academic background, the filtering stage should remove those as task candidates. These tests target a practical failure mode of text-generation systems: a polished but ungrounded schedule.

5.12 Model-evaluation protocol

For the workload classifier, Sourcetree records require an assignee and parseable creation date. Priority and issue type are mapped to complexity and a rule-based effort proxy. Apache records are converted to the same feature columns, with the project code as a pseudo-member. A stratified random split preserves outcome proportions. The labels are not equivalent: Fixed resolution for Sourcetree and the complement of delayed for Apache. Training-partition aggregates are merged onto both portions, but training rows also encode their own outcomes through completion-rate and unresolved-count aggregates. Unseen identities receive defaults. This is a descriptive mixed-label experiment, not validated student suitability modelling.

After fitting the legacy XGBoost model, the evaluator reports accuracy, weighted F1 and a confusion matrix on 283 row-split hold-out issues. Those labels retain their source-specific definitions, and its five-fold CV is not leakage-free. A newer Sourcetree-only experiment uses one Fixed-at-export target, chronological 80/20 testing and as-of member histories. It scores 31.94% accuracy and 0.2561 weighted F1 on 216 later issues, below the 77.31% majority baseline. Issue age and unresolved-at-export censoring still limit this target, and no student outcomes are present.

The effort-regressor experiment is separate. Its inputs come from 580 China and Desharnais project records, with 20% held out. MAE and RMSE are measured on log(1 + effort), not directly in hours. The former 135.2% script value is exp(log MAE)-1, not measured MAPE; this dissertation reports separately recomputed original-hour MAPE and median absolute percentage error with their definitions. The T5-small experiment has only five hand-referenced examples compared by ROUGE. It does not establish general extractor performance or say anything reliable about a fine-tuned T5 model.

5.13 Allocation-baseline reproducibility

The paired baseline is designed so policy differences are not caused by different task samples. For each trial, the evaluator samples one 20-task set from the held-out rows and applies all four policies to it. The random policy provides an unconstrained reference. LPT tests how well a transparent least-loaded heuristic can balance the same work. Model-only assignment tests the danger of selecting the highest task–member

score without capacity checks. The hybrid adds the 1.40 guard and whole-task rebalancing. The historical-profile and cold-start scenarios distinguish the case in which stored member statistics are available from the case in which they are all defaults.

The reported mean CV is computed from four member-hour totals in each trial, then averaged across 100 trials. Spread is maximum minus minimum planned hours; overload rate is the fraction of trials above the 1.40 load ratio. These measures quantify planned fairness, not actual expertise. The evaluator now retains trial-level records and paired hybrid-minus-LPT CV differences. Its observed 2.5th–97.5th percentile range is −1.25 to 3.16 percentage points for historical profiles; it is descriptive over resampled issues, not an independent-team confidence interval. No supervisor-authored baseline or student-outcome comparison was performed.

5.14 Evidence and academic constraint tests

Progress testing must examine state transitions, not just HTTP return codes. A submission first checks that the project has a schedule and that the student is eligible for the document task. A valid file receives a pending-review record and a notification; the same bytes submitted again should be rejected. A reviewer can open the stored file, and approval, rejection or changes-requested should create an audit event with the reviewer and comments. For GitHub, a valid signature but missing task key may prove webhook transport without providing task-linked progress; an invalid signature must be rejected before any trusted event is written. Duplicate deliveries and commit SHAs should not add repeated progress.

The synthetic 30-person panel and eight-hall workbooks were checked for required columns, role distribution and parseable availability. They provide enough candidates for a four-person panel on paper, but do not represent real staff willingness or every conflict case. Separate database-backed tests verify that an undersized hall is rejected in draft scheduling, rescheduling and publication. Missing-role, occupied-slot and rollback scenarios still require broader service-level testing with realistic group and institutional policies.

5.15 What the reported tests do not establish

The older 60-test claim has no retained raw output and is not counted as current evidence. The 11 focused PostgreSQL checks establish selected assignment, versioning, hall-capacity, History, deadline and synthetic local document/webhook behaviours. They do not test every negative review decision, all UI states, direct Team Formation or Quality/Risk payload exchange, or real-cohort outcomes. PDF fixtures are not scanned or varied real proposals; panel sheets are synthetic; and the GitHub test repository uses known task keys. These limits accompany each reported result.

5.16 Recommended acceptance criteria for a later field pilot

Before a real-cohort trial, the team should define an independent manual reference. Two academic reviewers could mark the actionable tasks in each consented proposal, resolve disagreements and measure extraction precision, recall and source-location accuracy. A supervisor could then estimate task hours and record why the algorithmic plan is accepted or edited. Actual completion dates and review decisions would permit calibration of the risk labels, rather than reusing the current capacity ratio as a probability. For workload allocation, compare LPT, hybrid and supervisor-authored plans on both fairness and later task outcomes, using group-level splits so the same team is not in training and test sets.

For the academic scheduler, acceptance should include the proportion of events scheduled without manual repair, the number of constraint violations after an independent calendar check and the time spent by staff resolving unscheduled cases. For the progress tracker, acceptance should include false task-key matches, duplicate-webhook handling, document-review turnaround and student understanding of rejection comments. No numeric threshold is asserted here because institutional requirements and realistic baseline measurements have not yet been collected. Recording these measures prospectively would make the next evaluation substantially stronger than a demonstration with synthetic fixtures.

5.17 Reproducibility of the predictive experiments

Reproducibility requires more than quoting an accuracy value. The workload experiment combines 1,078 Sourcetree and 333 Apache records, applies a seeded

stratified 80/20 row split, and stores the fitted XGBoost artifact. Its 283-row hold-out produces the reported confusion matrix, accuracy and weighted F1. These values are reproducible for that prepared dataset, but the two source labels have different meanings, only five Sourcetree assignee IDs are present, and five Apache project codes stand in for people. Each training row contributes its own label to the aggregate features used to train the model; the full prepared table also enters five-fold CV without rebuilding aggregates within each fold. This is not a leakage-free estimate of new-student or future-project performance.

The effort experiment uses 580 China and Desharnais project records and evaluates 116 held-out rows. Its target is log1p(effort hours): MAE 0.8554 and RMSE 1.0614 are log-scale errors, not hours. The older script prints 135.2% from exp(log MAE)-1; this is a transformation of one aggregate statistic, not a measured mean percentage error, and should not be cited as MAPE. Recomputing mean(|inverse-prediction minus actual hours| / actual hours) on the same hold-out gives about 152.6% MAPE; the median absolute percentage error is about 61.8%. These additional numbers are highly sensitive to the distribution of project hours. The model is a preliminary planning input, not a validated deadline predictor. A later study needs an explicit baseline and a student-project dataset.

5.18 Independent outputs for the allocation comparison

The four allocation policies are applied to the same sampled task sets within each paired trial. This design controls one major source of variation: a model-only run cannot appear worse merely because it received harder tasks than LPT. The evaluator reports both a distribution statistic, CV, and a threshold statistic, the percentage of trials with a max/min load ratio above 1.40. Mean hour spread gives the reader a sense of scale that CV alone hides. The historical-profile and cold-start scenarios are analysed separately because a learned score can depend heavily on member-history inputs that a new student group does not possess.

These trials still have important design limits. The four members are evaluation identities drawn from issue data or assigned defaults; they are not four real students collaborating over a semester. Tasks are sampled from a held-out issue pool rather than generated from one coherent proposal with shared dependencies. Planned hours are

assessed immediately after allocation, so no later rework, illness, missed milestone or supervisor override enters the measurement. The trial establishes relative balance of the simulated policies under its inputs. It does not establish that a hybrid-assigned team would finish a research project faster or produce better work.

5.19 Traceability tests for the document pathway

A suitable integration test constructs a known group, a known member and a task marked as requiring a document. Loading eligible tasks should show that task to the member and should not show another member's restricted task. The test then uploads a small valid PDF, checks that the response includes a submission version and pending status, and confirms that the stored hash equals the bytes sent. Repeating the identical bytes should be rejected as a duplicate; changing the file should create a new reviewable version. A file renamed to .pdf but with a wrong signature should fail independently of the extension check. These cases validate identity, storage and evidence semantics, not academic quality of the document.

Review tests should check both positive and negative decisions. The focused local run covered one approved PDF submission and a signed, task-keyed webhook, with persisted review and task-progress records. Rejection without a comment, unauthorised reviewer access, malformed files, duplicate delivery and unmatched-commit paths remain separate acceptance cases unless their execution logs are retained. The local signed payload is not a fresh GitHub.com delivery, and these checks do not amount to a production student-login test.

5.20 Webhook and event-scheduler negative cases

The webhook's decisive negative cases are an absent secret configuration, a missing or invalid X-Hub-Signature-256 header, an unregistered repository and a repeated delivery. None should create trusted task progress. The positive case uses a registered repository, a valid signature and a commit message containing an existing task key; it should create a linked activity record but not mark the task fully complete. A second positive transport case with no task key should remain unmatched or flagged. Checking only for a green GitHub delivery would miss all of these semantic distinctions.

For academic events, tests should force each failure mode independently: no eligible Lecturer, overlapping staff booking, overlapping hall booking, a short time window and an undersized hall. The expected result is an unscheduled record or rejected edit with a reason. The current database-backed regression suite includes capacity checks in generation, rescheduling and publication. Other negative cases and publication rollback still need comprehensive service-level and institutional acceptance testing.

5.21 Visual and document review protocol

The research artifact includes both a browser interface and this written report. Visual inspection should cover a new Gantt before balancing, the same project after balancing, an older history version, the task-detail evidence panel, a large searchable group list and the academic event draft. Dark and light themes need separate inspection because readable colours in one mode do not guarantee readable text or borders in the other. For the dissertation itself, the editable Word file should be opened in Microsoft Word, fields updated, and a PDF export inspected for page breaks, table overflow, figure legibility, page numbering and reference completeness. A programmatically counted page total alone is not a formatting check.

5.22 Evidence status and reporting rule

Numerical results in this dissertation come from saved training or offline-evaluation outputs, a stated recomputation on the same hold-out, and the retained 11 October 2026 focused database-test record. The WBS count and sample workbooks remain controlled fixtures. The 11 checks cover selected live scheduling, History, deadline, hall-capacity and synthetic local PDF-review/webhook paths; they do not validate every API, cross-component deployment or real student outcome. Proposed negative tests outside this focused suite remain acceptance criteria. Any later code or dataset change requires a fresh reproducible run before submission.

6 RESULTS AND VALIDATION

6.1 Predictive and extraction results

Table 6.1 presents observed numerical outputs and the exact bounds of their interpretation.

Table 6.1. Observed numerical outputs and the exact bounds of their interpretation.

Measure	Verified value	Interpretation
Legacy JIRA row-split accuracy	81.27%	Mixed proxy label and history leakage; not student suitability
Legacy JIRA weighted F1	0.815	Same historical experiment and validity limits
Five-fold accuracy	76.47% ± 0.95 pp	Aggregates reused across folds; not leakage-free
Sourcetree chronological accuracy / F1	31.94% / 0.2561	216 later issues; below 77.31% majority accuracy
Effort log MAE / RMSE	0.8554 / 1.0614	Log-scale error; high real-hour uncertainty
Effort original-hour MAPE	152.6%	Recomputed on same 116-row hold-out; not the script's 135.2% transform
Controlled WBS recovery	73 / 75 (97.3%)	Known entries in synthetic PDFs; precision unmeasured

The classifier confusion matrix is [[103, 6], [47, 127]] for source-dependent class 0 and class 1, with recalls 0.945 and 0.730. Sourcetree labels are based on Fixed resolution; Apache labels are based on absence of a delayed flag, so a single 'on-time student task' label would be false. The effort script's 135.2% equals exp(0.8554)-1 and is not measured MAPE. Original-hour MAPE was recomputed as the mean of absolute prediction error divided by actual effort over the same 116 hold-out rows; the result is about 152.6%, with median absolute percentage error about 61.8%. The extractor returned 24, 24 and 25 of 25 known entries in three synthetic proposals, with false-positive precision not evaluated.

6.2 Allocation baselines

Figure 6.1 shows historical-profile allocation baseline: mean workload cv over 100 paired trials.

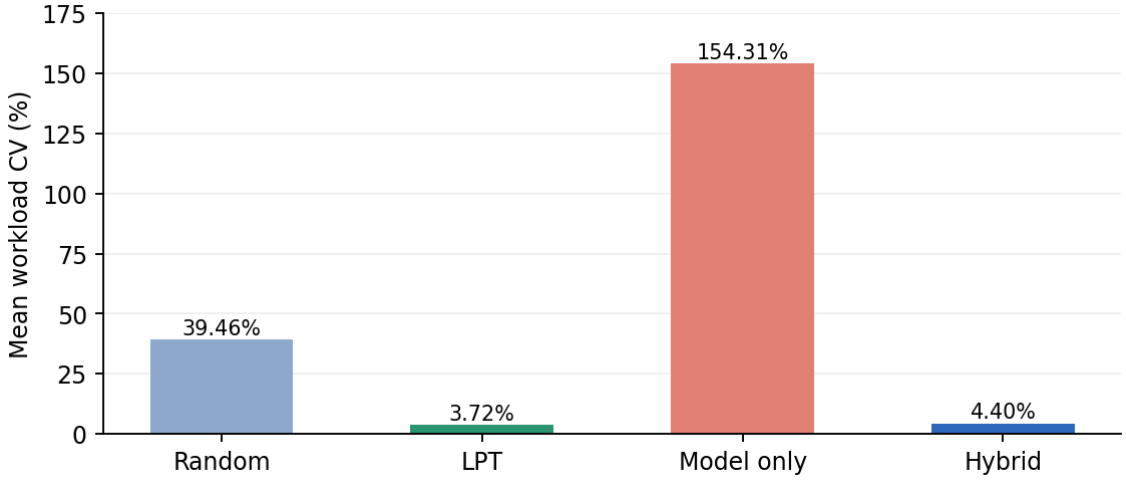


Figure 6.1. Historical-profile allocation baseline: mean workload CV over 100 paired trials.

Table 6.2 presents historical-profile and cold-start allocation baselines over 100 paired trials per scenario.

Table 6.2. Historical-profile and cold-start allocation baselines over 100 paired trials per scenario.

Policy	Historical CV	Overload trials	Cold-start CV
Random	39.46%	98%	42.35%
LPT	3.72%	0%	3.68%
Model only	154.31%	100%	167.40%
Hybrid	4.40%	0%	4.28%

In the offline resampling experiment, the score-only policy concentrated work. The guarded hybrid averaged 4.40% CV with no observed overload in 100 historical-profile trials; LPT averaged a lower 3.72% CV. These are planned-hour fairness results, not evidence that proxy ranking improved student suitability or completion. Trial-level records and paired descriptive differences are now saved; the historical-profile hybrid-minus-LPT CV difference averaged +0.68 percentage points. No supervisor-authored-plan comparison was performed.

6.3 Operational findings

The current implementation separates provisional Gantt dates from final student ownership: internal capacity lanes do not appear as approved owners, and a unique authenticated LPT route saves the balanced version. Focused database tests cover ownership persistence and rollback. Document and GitHub records retain separate receipt, activity and approval states. Academic event selection checks roles, time conflicts and a configured hall-seat requirement. These are selected implemented and tested behaviours, not a completed end-to-end institutional validation.

7 DISCUSSION

7.1 Interpreting the workload classifier

The 283-row confusion matrix records 103 correct class-0, six class-0 predicted class-1, 47 class-1 predicted class-0 and 127 correct class-1 cases. Class membership combines non-equivalent Sourcetree and Apache definitions. It is therefore inappropriate to translate a false positive into 'the wrong student was recommended' or a true positive into a successful student task. The matrix only describes predictions of the mixed issue-status proxy under this split.

The five-fold mean accuracy of 76.47% is 4.80 percentage points below the single 81.27% row-split value. Neither should be presented as a reliable deployment estimate: the CV features were prepared before its folds, training rows encode their own outcomes in aggregate features, and only five Sourcetree assignee IDs plus five Apache project pseudo-members are represented. The low fold-to-fold standard deviation concerns those prepared folds only. Grouped, temporal and student-cohort testing is required before any claim about actual team-member recommendations.

7.2 Why the hybrid requires constraints

Table 7.1 presents additional historical-profile allocation measures (100 trials).

Table 7.1. Additional historical-profile allocation measures (100 trials).

Policy	Mean spread (h)	Guard overrides / trial	Rebalance moves / trial
Random	187.02	0	0
LPT	16.68	0	0
Model only	657.78	0	0
Hybrid	19.44	13.83	1.92

The model-only policy's 657.78-hour mean spread shows why unconstrained proxy scoring is unsuitable as a fairness rule. In the offline evaluator, the hybrid records 13.83 guard overrides and 1.92 accepted whole-task moves per historical-profile trial. These experimental safeguards are not part of the default live LPT route. Its final 19.44-hour spread is near but above LPT's 16.68 hours. No project quality or completion-time benefit was measured.

Cold-start trials reinforce the same interpretation. LPT and hybrid average 3.68% and 4.28% CV, with no overload violations, despite all four member profiles starting from the same defaults. Random assignment violates the overload ratio in 99% of cold-start trials and model-only assignment in 100%. A deterministic load rule is therefore especially important when historical data are sparse. If a supervisor needs to reserve a specialist task for a particular member, the system should make that override visible and recalculate fairness, rather than hide the constraint inside an unexplained model score.

7.3 Extraction and evidence examples

The three proposal fixtures give a narrow but useful extraction check: two produced 24/25 known entries and one produced 25/25. The missed items may arise from conservative acceptance rules or wording outside the recognised patterns. A rule that never misses an explicit task might also accept many background sentences as work, so future evaluation should report precision and recall on a separately annotated set of real proposals. The present 97.3% is correctly called recovery of explicit synthetic entries, not general task-extraction accuracy.

The R26-IT-095 evidence pack illustrates how this plan can be exercised. Its manifest maps 27 source-proposal tasks to document, GitHub or manual evidence. Five prepared example documents correspond to task IDs 002 (Literature Review), 012 (User Requirements), 013 (Functional Requirements), 026 (Pilot Study) and 027 (Report Writing). Thirteen prepared commits carry task-key references. These are fixtures for testing the user journey; they are not independent proof that the real project completed those tasks or that its supervisor accepted them. The actual review state must come from the submission and approval records in the running database.

7.4 Academic scheduling and integration interpretation

The 30-person and eight-hall fixtures confirm that required workbook columns, roles, dates and time ranges can be parsed. Focused database tests additionally check hall-seat rejection during scheduling, rescheduling and publication. These observations do not establish that every panel role, conflict, holiday or real staff preference is handled

in an institutional run. Formal acceptance requires a live multi-group scheduling cycle and an independent constraint check against actual policies.

Together, the results support a conservative stance. The prototype contains connected planning, evidence and timetable structures, and selected assignment and hall-capacity paths passed focused tests. Offline algorithms have measurable behaviour, but weak proxy validity and absent real-cohort outcomes preclude a claim of improved student performance or full institutional integration. Deliverable approval, ambiguous evidence, timetable publication and high effort estimates require authorised human review.

7.5 Threats to validity

The legacy experiment's construct validity is limited by mixing Sourcetree Fixed resolution with Apache not-delayed status; Apache uses project pseudo-members and only five Sourcetree assignee IDs are present. Its internal validity is limited by self-inclusive training aggregates, row-level identity reuse and cross-validation on precomputed features. The newer Sourcetree-only chronological rerun reduces some leakage but is affected by issue age and unresolved-at-export censoring and performs below the majority baseline. External validity is limited because allocation trials resample issues and other artifacts are synthetic. Focused operational tests cover only selected paths. No student cohort, supervisor-authored baseline or longitudinal outcome was evaluated.

Operational risk is also not a calibrated probability; NSGA-II and Monte Carlo endpoints have not been rigorously benchmarked in this work; and the optional zero-shot T5 check had only five examples (ROUGE-1 0.2621; ROUGE-L 0.2447). The system processes potentially personal documents and GitHub identities, so a production deployment needs consent, restricted access, secret rotation, retention rules and institutional data-governance approval.

7.6 Error analysis of the classifier

Figure 7.1 shows legacy mixed-label row-split confusion matrix; rows are actual labels and columns are predictions.

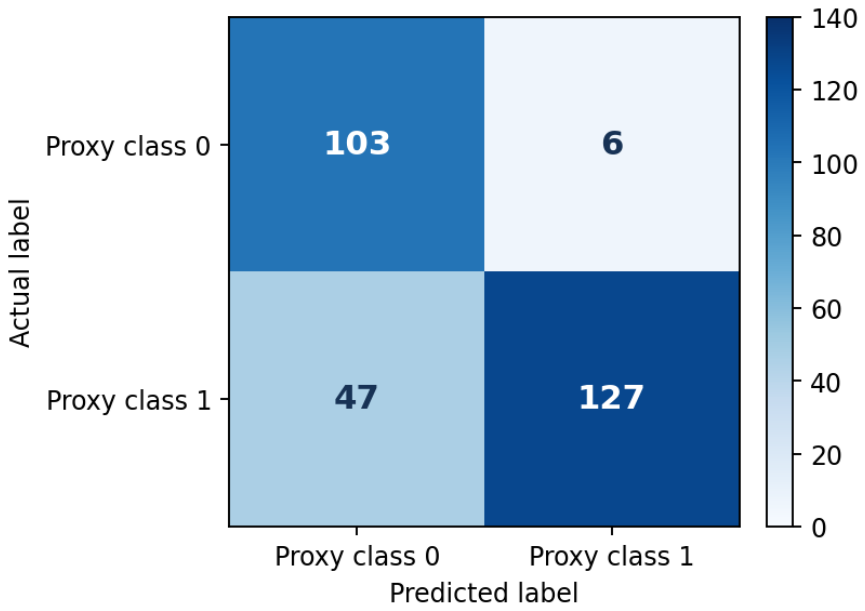


Figure 7.1. Legacy mixed-label row-split confusion matrix; rows are actual labels and columns are predictions.

The confusion matrix gives a more precise interpretation of the 81.27% row-split accuracy. Six of 109 class-0 rows were predicted class 1, while 47 of 174 class-1 rows were predicted class 0. Class 0 means not Fixed in Sourcetree or delayed in Apache; class 1 means Fixed in Sourcetree or not delayed in Apache. The class recalls differ materially, and neither class denotes supervisor-rated student suitability. The model's training features additionally encode outcomes from the same training rows, while row-level splitting can reuse identities across partitions. A credible follow-up needs consistent labels, out-of-fold or prior-history features, a grouped/time-aware split, and supervisor-rated candidate pairs.

7.7 Interpreting the fairness numbers

The historical-profile CV gap between hybrid and LPT is 0.68 percentage points (4.40% versus 3.72%). That difference is small compared with random and model-only assignment, but it is in LPT's favour. The hybrid's zero observed overload rate in 100 trials is an empirical observation, not a proof that every possible task set satisfies

the 1.40 ratio. The pre-assignment guard's current-load comparison and the indivisibility of tasks both prevent such a universal claim. The defensible conclusion is that adding constraints rescued the learned ranking from severe concentration in the controlled experiment while preserving balance close to the simple baseline.

The model-only result is particularly informative because it exposes what the classifier does not know. Its score may favour an assignee with historical completion patterns even when that member's planned load has become excessive. Without an explicit capacity variable that is calibrated to student availability, a ranking can optimize the wrong local quantity. The hybrid addresses this failure mode algorithmically, but the experiment does not show whether any loss of balance buys better expertise matching. Future evaluation should record a supervisor-rated suitability score or observed rework for each assignment, rather than treating lower CV as the sole project outcome.

7.8 Interpreting the extraction result

Seventy-three recovered tasks out of 75 explicit synthetic WBS entries is useful as a regression target for the PDF parsing path. It also illustrates why the unit of evaluation must be stated: these are list entries in designed fixtures, not all action statements in a natural proposal. The two missed items should be inspected by their source format and filter reason; otherwise a single recall number cannot guide an improvement. A more permissive parser could recover both yet introduce false tasks from headings or references. The next evaluation therefore needs paired precision and recall, a false-positive catalogue and an annotation protocol on independent documents.

7.9 Component-level answer to the research questions

RQ1 has a bounded answer: live allocation now uses a deterministic LPT baseline, while an offline guarded hybrid produced near-LPT fairness on sampled issues and a time-aware JIRA proxy performed below the majority baseline. There is no demonstrated improvement in student assignment quality. RQ2 has a narrow positive answer: explicit WBS entries were recovered in three structured synthetic PDFs; precision on general proposals was not measured. RQ3 has an implementation answer supported by focused tests for balanced-version persistence and hall-capacity checks, but full cross-component and real-cohort workflow validation remains outstanding.

These answers explain why the component can be presented as a working research prototype without overstating it as a validated institutional decision system. It integrates several previously separate actions into one traceable project record, but its strongest evidence remains local model and fixture experiments. The distinction is essential in a viva: a demonstration of the Gantt and review screen shows that the workflow exists; the experimental tables show how particular algorithms behaved; neither by itself proves improvement in real cohort outcomes.

7.10 Summary of individual contribution

The component-level artefacts cover proposal-linked task extraction, versioned scheduling with live LPT assignment, document and GitHub evidence review, and academic-event planning. The student's exact personal implementation, evaluation, writing and integration responsibilities within the shared IPMS project must be confirmed against the team's approved contribution statement before this section is treated as a formal individual-authorship claim.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value proposition and target users

The immediate target environment is a faculty supervising many undergraduate groups. A useful institutional deployment would combine three benefits: a consistent first-draft plan from a proposal, a searchable view of group workload and evidence, and a timetable that checks staff and room constraints. The prototype may reduce repetitive data entry and make review decisions easier to audit, but this report has not measured staff time saved or financial return. It would be premature to present a revenue model or quantified cost reduction from the available evidence.

8.2 Adoption and evaluation path

A phased adoption path is more defensible. First, keep the tool as an advisory planner with staff checking each extracted WBS and effort estimate. Second, pilot the progress portal with consented groups and measure document-review time, correction rate, and rejected or mislinked GitHub events. Third, test event scheduling against real hall calendars and staff availability, including absences and policy exceptions. Only after security review and operational support are in place should a faculty treat published timetable output as authoritative. This staged approach could collect consented student-team data for a new, properly labelled assignment model; the present mixed JIRA proxy classifier has not been shown to transfer to these users.

8.3 Data rights, governance and sustainability

The local Sourcetree export, Apache-project summary and Desharnais copy need their acquisition records and reuse terms checked before raw-data redistribution. The China subset matches a published Zenodo record, but matching content is not by itself a licence grant. Any institutional pilot would also need stable hosting, backups, access logs, student identity, consent and a retention policy. The project may align with SDG 4 by supporting educational project supervision, but educational impact has not been measured.

9 CONCLUSION AND FUTURE DIRECTIONS

The component implements proposal-linked planning, a live LPT workload transition, evidence records and constrained academic scheduling, with selected behaviours verified in an isolated database. The legacy 81.27% mixed JIRA-proxy row-split result is not student suitability accuracy; the newer chronological Sourcetree proxy scored 31.94%, below a 77.31% majority baseline. The 73/75 extraction value is recovery on synthetic PDFs, not general accuracy. The offline hybrid's 4.40% CV is slightly worse than LPT's 3.72%; it is not the live policy. Effort estimates have high original-hour errors. These limits define the next research and operational validation steps.

Future work should collect consented undergraduate project histories, validate with grouped and temporal splits, compare plans with supervisor-authored schedules and measure actual deadline adherence. Effort prediction and schedule-risk calibration need improvement before their values can be treated as reliable forecasts. A real user study should also test whether the linked Gantt, evidence review and academic scheduler reduce administrative work without reducing human oversight.

9.1 Summary of achieved objectives

The component contains proposal-grounded task extraction, dated Gantt generation, a live LPT-based balanced-version transition, evidence-review records and academic-timetabling functions. A separate offline evaluator measures four allocation policies, including a guarded hybrid. The individual contribution is the evidence-aware workflow, its implementation checks and bounded evaluation, not a claim that the JIRA classifier measures student suitability or that every institutional deployment condition is validated.

9.2 Limitations and next research steps

The remaining technical priorities are broader cross-component integration tests, institutional authentication for students, independent timetable checks, and operational security review. Methodological priorities are consented student-task labels, prospective group-level validation, comparison with supervisor-authored plans, and longitudinal outcomes. A chronological Sourcetree-only proxy rerun and trial-level allocation output now exist, but neither supplies student-suitability evidence. The

effort model needs a strong baseline and calibration; extraction needs precision and recall on independently annotated proposals. This remains a research prototype, not evidence of improved real-cohort outcomes.

REFERENCES

[1] I. Taboada et al., “Artificial Intelligence Enabled Project Management: A Systematic Literature Review,” Applied Sciences, vol. 13, no. 8, art. 5014, 2023. doi: 10.3390/app13085014.

[2] M. Jørgensen and M. Shepperd, “A Systematic Review of Software Development Cost Estimation Studies,” IEEE Transactions on Software Engineering, vol. 33, no. 1, pp. 33–53, 2007. doi: 10.1109/TSE.2007.256943.

[3] Z. Kurtanović and W. Maalej, “Automatically Classifying Functional and Non-Functional Requirements Using Supervised Machine Learning,” Proc. IEEE Requirements Engineering Conference, pp. 490–495, 2017. doi: 10.1109/RE.2017.82.

[4] Y. Luan et al., “Multi-Task Identification of Entities, Relations, and Coreference for Scientific Knowledge Graph Construction,” Proc. EMNLP, pp. 3219–3232, 2018. doi: 10.18653/v1/D18-1360.

[5] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[6] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” Proc. ACM SIGKDD, pp. 785–794, 2016. doi: 10.1145/2939672.2939785.

[7] A. B. Kahn, “Topological Sorting of Large Networks,” Communications of the ACM, vol. 5, no. 11, pp. 558–562, 1962. doi: 10.1145/368996.369025.

[8] R. L. Graham, “Bounds on Multiprocessing Timing Anomalies,” SIAM Journal on Applied Mathematics, vol. 17, no. 2, pp. 416–429, 1969. doi: 10.1137/0117039.

[9] J. Anvik, L. Hiew and G. C. Murphy, “Who Should Fix This Bug?,” Proc. ICSE, pp. 361–370, 2006. doi: 10.1145/1134285.1134336.

[10] E. Kalliamvakou et al., “The Promises and Perils of Mining GitHub,” Proc. MSR, pp. 92–101, 2014. doi: 10.1145/2597073.2597074.

[11] G. Salton and C. Buckley, “Term-Weighting Approaches in Automatic Text Retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.

[12] W. Wang et al., “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” Advances in Neural Information Processing Systems, vol. 33, pp. 5776–5788, 2020.

[13] A. Schaerf, “A Survey of Automated Timetabling,” Artificial Intelligence Review, vol. 13, no. 2, pp. 87–127, 1999. doi: 10.1023/A:1006576209969.

[14] K. Deb et al., “A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II,” IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182–197, 2002. doi: 10.1109/4235.996017.

[15] N. Metropolis and S. Ulam, “The Monte Carlo Method,” Journal of the American Statistical Association, vol. 44, no. 247, pp. 335–341, 1949. doi: 10.1080/01621459.1949.10483310.

[16] J. Sayyad Shirabad and T. J. Menzies, The PROMISE Repository of Software Engineering Databases, University of Ottawa, 2005.

[17] Atlassian, "SRCTREEWIN-14001: Authentication failed when attempting Fetch command," Sourcetree for Windows Jira issue, 2023. [Online]. Available: https://jira.atlassian.com/browse/SRCTREEWIN-14001

[18] F. H. Yun, "China: Effort Estimation Dataset," Zenodo, v1, 2010. doi: 10.5281/zenodo.268446.

APPENDIX A: EVIDENCE AND CODE MAP

The following project-local files substantiate the implementation and numerical claims. Source-code comments may use legacy model names; estimator identity should be verified from the loaded artifact and training code. Table A.1 maps each evidence type to its project-local file.

Table A.1. Evidence and code map.

Evidence	Location
Current main scheduler and APIs	component_scheduling/main.py
Legacy workload classifier training	component_scheduling/train_workload_model.py
Chronological Sourcetree-only proxy experiment	component_scheduling/train_workload_model_v2.py
Chronological proxy results	component_scheduling/ieee_paper/workload_proxy_v2_results.json
Paired allocation experiment	component_scheduling/evaluate_workload_balancing.py
Allocation numeric summary	component_scheduling/ieee_paper/workload_baseline_summary.json
Paired trial-level allocation data	component_scheduling/ieee_paper/workload_baseline_trials.csv
Progress and review logic	component_scheduling/progress_tracking.py
Academic panel constraints	component_scheduling/academic_scheduler_service.py
Evaluation/claim audit	component_scheduling/ieee_paper/SOURCE_AUDIT.md
Controlled fixture validation	component_scheduling/test_data_pack/VALIDATION_REPORT.md
Focused database regression suite	component_scheduling/test_scheduling_remediation.py
11 Oct. regression	component_scheduling/Scheduling_Submission_Verification_2026-10-11.md
Synthetic-flow detail	component_scheduling/Scheduling_Fresh_Integration_Test_Evidence_2026-10-10.md
External commit audit	component_scheduling/External_Evidence_Audit_2026-10-10.md
Dataset provenance	component_scheduling/Dataset_Source_Provenance_Audit_2026-10-11.md

APPENDIX B: ACADEMIC INTEGRITY AND AI-USE DISCLOSURE

Draft disclosure for author review: OpenAI Codex assisted with drafting and editing parts of the dissertation, reorganising and formatting the document to the supplied SLIIT template, checking source-code claims, and comparing local dataset contents with public records. These tools were not a source of experimental observations. The author must verify all numerical results, citations, code descriptions and attributions, and add any other AI tools, prompts or affected sections required by the module's academic-integrity policy before signing the declaration.